



Пояснительная записка
к дипломной работе на тему:

**"Классификация произведений известных художников с помощью
компьютерного зрения"**

Автор: Загускина Юлия
Группа: DSU-22

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ.....	2
ВВОДНАЯ ЧАСТЬ.....	2
ПОСТАНОВКА ЗАДАЧИ.....	3
АНАЛИЗ ДАННЫХ.....	4
МЕТОДИКА РЕАЛИЗАЦИИ.....	12
ВЫВОДЫ.....	16
СПИСОК ИСТОЧНИКОВ.....	17

ВВОДНАЯ ЧАСТЬ

Творчество - это проявление высших способностей человека, высшая форма его деятельности. Живопись - это одно из самых выразительных искусств, которое позволяет человеку передать свои мысли, эмоции и взгляды на мир через цвет и форму. Каждый художник имеет свой уникальный стиль и подход к созданию произведений, что делает живопись настоящим выражением творчества. Живопись, являясь выражением визуального языка, существует с начала цивилизации (напоминающие хэштег линии, сделанные охрой на куске силкриты, возможно, были древнейшим в мире рисунком, написанным около 73 тысячи лет назад в пещере Бломбос в ЮАР). И вот мир достиг стадии, когда компьютеры создают картины. Искусствоведы, специализирующиеся на определённой эпохе, художнике или художественном направлении, могут определить кисти какого художника принадлежит картина или определить стиль картины... а может ли компьютер сделать то же самое?

В последние десятилетия искусственный интеллект и глубокое обучение стали неотъемлемой частью многих сфер человеческой жизни, включая компьютерное зрение и обработку изображений. Среди областей, использующих технологии машинного обучения, классификация изображений выделяется особой важностью и вот несколько примеров: создание тематических карт, оптическое распознавание символов (OCR), анализ медицинских изображений, топографическое моделирование, навигация автономных транспортных средств или мобильных роботов (в декабре 2023 г. Россия запустила первый беспилотный рейс паромы из Санкт-Петербурга в Калининград), визуальное наблюдение, автоматический контроль на производстве, задачи идентификации и т.д. Современные методы классификации изображений часто основаны на свёрточных нейронных сетях, которые демонстрируют высокую эффективность в распознавании и анализе визуальных данных. Одной из первых успешных архитектур для распознавания рукописных цифр была LeNet-5, разработанная Янном ЛеКуном в 1998 году.



Древнейший из известных наскальных рисунков был обнаружен в пещере в Индонезии на юге острова Сулавеси, которому как минимум 45,5 тысяч лет, нанесён красной охрой.



Картина «портрет Эдмонда Белами», созданная искусственным интеллектом, ушла с молотка за \$0,5 млн. в 2018 году. Работа была создана французскими студентами.

ПОСТАНОВКА ЗАДАЧИ

Последние несколько лет количество оцифрованных произведений изобразительного искусства растёт, что привело к необходимости разработки мультимедийных систем для архивирования, внесения и извлечения данных. Например, для онлайн-галерей, демонстрирующих живопись, существует необходимость разработать автоматизированные системы рекомендаций, которые смогут находить «похожие» картины, например, в жанре пейзажа или портрета, в стиле ренессанса или барокко, или картины определённого художника, которые пользователь хотел бы приобрести. И измерение визуального сходства между произведениями, в этих случаях, — это необходимая и важная вещь. Чтобы смоделировать сходство между картинами, нужно извлечь визуальные особенности картин и найти лучший подход к изучению метрики сходства.

На сегодняшний день существует большое разнообразие направлений, стилей, техник в живописи. Они не имеют точных границ, они взаимодействуют, перемешиваются. В своих произведениях авторы могут использовать несколько стилей и техник, создавая произведения, которые невозможно точно классифицировать. И это не может не представлять сложности для нейросетей: например, «похожие» картины в одном стиле, модель должна научиться «видеть» собственную технику выполнения каждого художника и объединяющие особенности техники выполнения, характерные для стиля.

Цель - научить модель предсказывать художника по картине, но не объекты на ней. Значит, модель должна научиться «понимать» особенный подход художника к цвету, форме, «уловить» уникальную технику каждого художника.

Задача - обучить нейронную сеть на классификацию художников по их картинам с точностью на тестовой выборке выше 80%.

Метрика качества — точность (accuracy) - распространённый эмпирический метод оценки точности классификации — это процент корректно классифицируемых примеров.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Данная мера не акцентируется на каком-то конкретном классе, оценивается ситуация в целом. В случаях, когда набор данных не сбалансирован, данная метрика может вводить в заблуждение, что крайне важно учитывать в медицинских исследованиях, где нужно быть точно уверенным, что метрика не выдаёт ложноотрицательный результат (false negative).

АНАЛИЗ ДАННЫХ

В 2021 было проведено исследование на таком же наборе данных «Best Artworks of All Time» с целью классифицировать картины по художникам. Авторы исследования - И.И. Евтушенко, А.В. Вечур¹ (ХНУРЭ, г. Харьков, Украина, 2021 г.).

Результатом исследования стала точность = 77.75% (macro f1-score = 0.722). В ходе исследования модели были проверены гипотезы по оптимальному разрешению входного изображения (112x112 / 224x224 / 448x448), по различным методам оптимизации (SGD, Adam, LAMB), различным методам изменения скорости обучения (константный, затухающий, косинусный), методам регуляризации для борьбы с переобучением (Dropout, Регуляризация Тихонова, Модификация данных), т. д. Результат (выше) показала модель с размерностью изображений 224*224, с применением поворота картины по вертикальной оси, алгоритмом оптимизации Adam с параметрами по умолчанию, с функцией потерь кросс-энтропия, с применением многозадачного обучения: задача регрессии, решаемая путем минимизации среднеквадратичного отклонения между предсказаниями и метками, метриками macro f1-score и accuracy. Обучение проводилось на 20 эпохах. Для теста использовали данные “Wiki-Art: Visual Art Encyclopedia”.

Описание данных.

В работе используется язык Python и библиотека PyTorch. Данные были взяты с платформы Kaggle², который, в свою очередь, был собран с сайтов wikipedia и [artchallenge.ru](https://www.artchallenge.ru).

Данные содержат 3 файла:

artists.csv - набор данных с информацией о каждом художнике (id, name, years, genre, nationality, bio, wikipedia, paintings),

images.zip - коллекция изображений (в полном размере), разделенных по папкам и последовательно пронумерованных,

¹ <https://openarchive.nure.ua/bitstreams/39c7f797-a8f2-41ee-9394-f52a55af286e/download>

² <https://www.kaggle.com/datasets/ikarus777/best-artworks-of-all-time/data>

resized.zip - та же коллекция, но размеры изображений были изменены и извлечены из структуры папок.

Набор данных состоит из 8446 изображений картин разной размерности 50 известных художников (рис.1).

Данные из файла artists.csv будут использоваться для получения общего представления о данных, например, количество картин и художников. Проверены данные на пропуски, не обнаружены.

Данные из папки images.zip будут использоваться для обучения и тестирования. Тип данных: изображение в формате jpg.

	id	name	years	genre	nationality	bio	wikipedia	paintings
44	44	Joan Miro	1893 – 1983	Surrealism	Spanish	Joan Miró i Ferrà (; Catalan: [ʒuˈam miˈro j f...	https://en.wikipedia.org/wiki/Joan_Miró	102
19	19	Albrecht Dürer	1471 - 1528	Northern Renaissance	German	Albrecht Dürer (; German: [ˈʔalbʁɛçt ˈdyːʁ]; ...	http://en.wikipedia.org/wiki/Albrecht_Dürer	328
27	27	Diego Velazquez	1599 - 1660	Baroque	Spanish	Diego Rodríguez de Silva y Velázquez (Spanish:...	http://en.wikipedia.org/wiki/Diego_Velázquez	128

Файл artists.csv с данными о каждом художнике

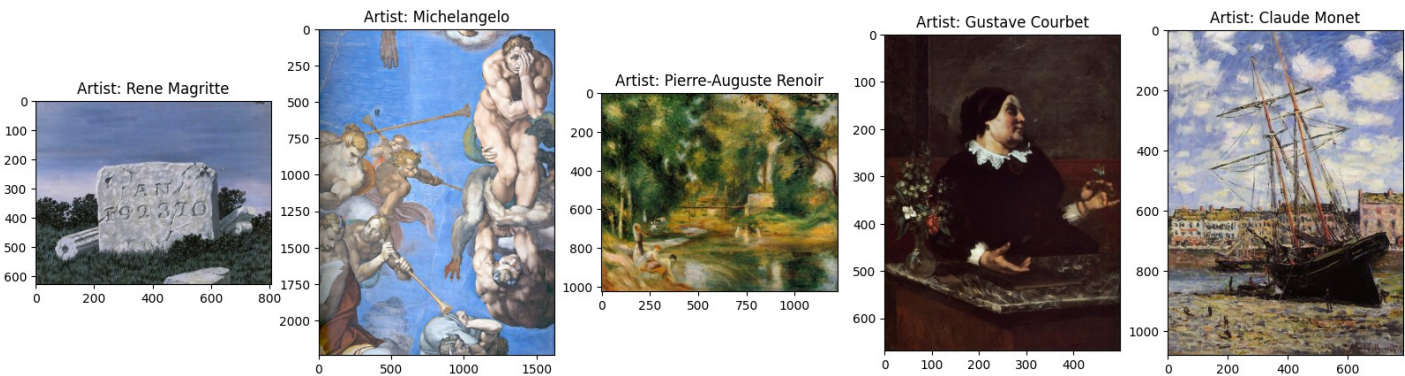


Рис. 1 5 картин случайных художников

В данных имеется несбалансированность классов (рис.2): 877 картин Ван Гога против 24 картины Джексона Поллока. Несбалансированность классов, как правило, создаёт проблемы при решении задач классификации, так как построенная на таких данных модель толком не выучит редкие классы, а потому не научится их отличать и, с большей вероятностью, будет присваивать метку большего класса на тестовых данных (переобучение под большой класс).

Хотя существуют различные способы, результаты исследования³ по устранению дисбаланса классов при обучении моделей свёрточных нейронных сетей на различных наборах данных (MNIST, CIFAR-10, ImageNet) показали, что почти во всех случаях лучшей стратегией было пересэмплирование / передискретизация (oversampling) редких классов, т.е. увеличение частоты просмотра моделью изображений из редких классов во время обучения. И это не привело к переобучению.

³ <https://arxiv.org/abs/1710.05381>

Поэтому для балансировки классов был применён `WeightedRandomSampler`⁴, чтобы модель видела изображения всех классов примерно одинаковое количество раз.

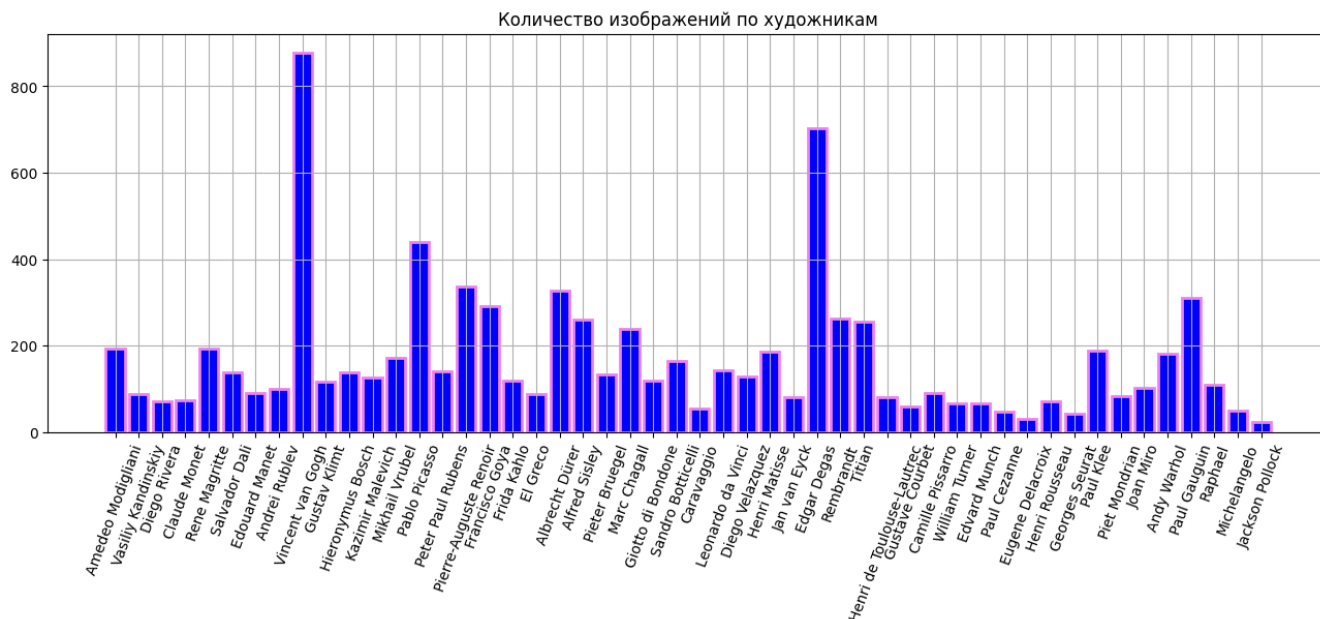
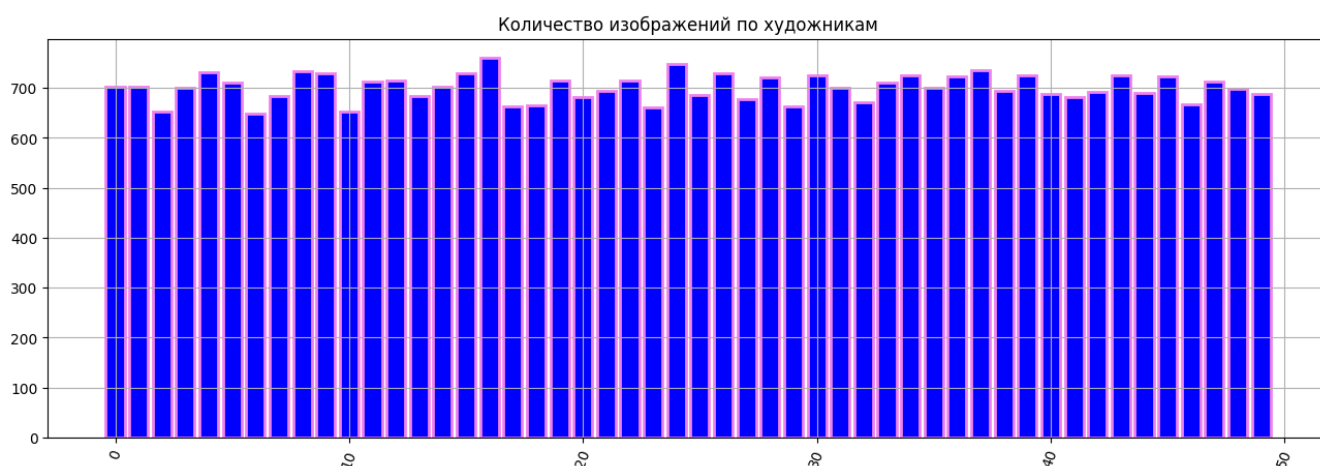


Рис. 2 Распределение картин по художникам

Была вычислена частота появления каждого класса в датасете для выставления веса для выравнивания сэмплов: умножается вес для редких классов пропорционально их редкости, чтобы высокий вес заставлял модель учиться на них и чаще видеть изображения этих классов, чтобы они перестали быть редкими.

Результат применения `WeightedRandomSampler`.



Разделение изображений на train-выборку (тренировочную) и test-выборку (тестовую) в соотношении 80 к 20 для каждого класса с соблюдением пропорции классов: количество для обучения — 6.756 изображений, для теста — 1.690.

В каждой выборке (train, train без аугментации, test) представлены все 50 классов.

Модель будет обучаться:

⁴ <https://www.kaggle.com/code/wervlad/baseline-2fc-augmentations>

- на данных без аугментации, с применением нормализации: среднее = [0.485, 0.456, 0.406], стандартное отклонение = [0.229, 0.224, 0.225] и приведение к одной размерности входных данных 224*224;

- на данных с аугментацией, также с применением нормализации, приведением изображений к одной размерности, также были применены преобразования: переворот изображения по горизонтали случайным образом (RandomHorizontalFlip), переворот изображения по вертикали под случайным углом (RandomVerticalFlip), преобразование случайной перспективы (RandomPerspective).

Применять другие преобразования, например, преобразование в оттенки серого или изменение контраста, не целесообразно, так как цвет, яркость, насыщенность являются важными признаками техники художника и, изменяя их, получится некорректный результат, модель может не «поймать» закономерности.

Для обучения выбраны 3 модели - mobilenetv2, mobilenetv3, efficientnetv2-s.

Обучаться каждая модель будет на 20 эпохах на графическом процессоре GPU, размер пакета данных (batch) — 20. Количество эпох и пакет данных были выбраны такими из-за небольших технических ресурсов.

Была выбрана функция потерь, которая вычислит, насколько предсказанный моделью результат далёк от истинных меток, — CrossEntropyLoss, которая оценит разницу между двумя вероятностными распределениями, одно из которых является вырожденным, функция включает в себя LogSoftmax и NLLLoss (Negative Log Likelihood Loss). На вход принимает логиты. LogSoftmax улучшает числовую производительность и оптимизацию градиента нежели Softmax. На вход NLLLoss передаются логарифмические вероятности.

Если результат (loss) велик, это значит, что и разница между двумя распределениями велика, если же мал, то распределения похожи друг на друга.

$$\text{Loss} = - \sum_{j=1}^K y_j \log(\hat{y}_j)$$

where k is number of classes in the data

Был выбран алгоритм оптимизации Adam (ADaptive Momentum) с шагом 1e-4, так как оптимизатор сочетает в себе идеи Momentum и RMSProp: происходит накопление значения градиента для определения направления и имеет адаптивную скорость обучения (learning rate) для каждого параметра модели на каждом шаге и, если параметр со слабыми изменениями, то скорость обучения завышается, а если параметр с сильными изменениями, то занижается. Скорость обучения адаптируется на основе среднего первого момента (среднего значения) и также использует среднее значение вторых моментов градиентов. Momentum позволяет повысить устойчивость и избегать маленьких локальных минимумов, седловых точек, «перевалить через стенку». Преимущества этого оптимизатора, помимо прочего, это простая реализация, вычислительная эффективность, небольшие требования к памяти, решение для задач с очень шумными или разреженными градиентами, меньше настроек для гиперпараметров.

$$v_{k+1} = \beta_1 v_k + (1 - \beta_1) \nabla f(x_k)$$

$$G_{k+1} = \beta_2 G_k + (1 - \beta_2) (\nabla f(x_k))^2$$

$$x_{k+1} = x_k - \frac{\alpha}{\sqrt{G_{k+1} + \epsilon}} v_{k+1}.$$

Как правило, в этом алгоритме подбирают лишь один гиперпараметр α – learning rate. Остальные же: β_1 , β_2 и ϵ – оставляют стандартными и равными 0.9, 0.99 и 1e-8 соответственно. Подбор α – это искусство.

MobileNet – это модели, разработанные специально для мобильных платформ и использующие оптимизированные алгоритмы для ускорения работы сети. Используются для задач компьютерного зрения таких, как классификация изображений, семантическую сегментацию, распознавание объектов и обнаружение лиц в мобильных устройствах.

Архитектура MobileNetv2⁵, предобученная на данных ImageNet (1000 классов), вес модели 13.6M.

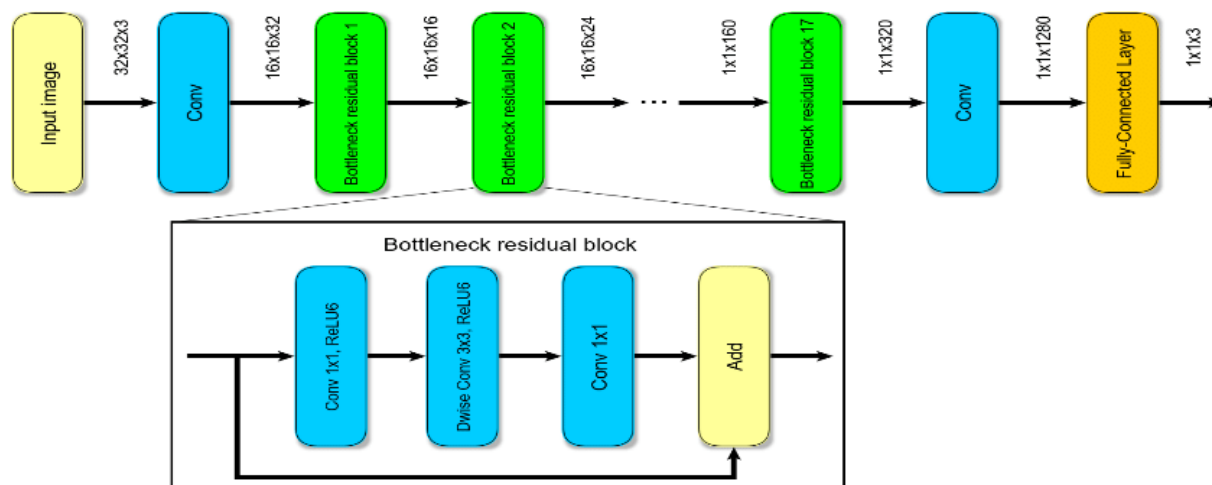


Рис. 3 Базовая архитектура mobilenetv2

MobileNetv2 основывается на идеях MobileNetv1, используя отдельную по глубине свёртку (depthwise separable convolution) в качестве эффективных строительных блоков. Однако v2 вводит в архитектуру две новые функции: 1) линейные узкие места (bottlenecks) между слоями и 2) короткие соединения (shortcut или skip connections) между узкими местами. Структура модели на рис.3

Узкие блоки кодируют промежуточные входные и выходные данные модели, в то время как внутренний слой инкапсулирует способность модели преобразовывать из представлений более низкого уровня, таких как пиксели, в дескрипторы более высокого уровня, такие как категории изображений. Наконец, как и в случае с традиционными остаточными соединениями, короткие соединения (shortcuts) позволяют ускорить обучение и повысить точность, так как соединяют напрямую узкие блоки.

Bottleneck layers состоит из 3 слоёв: 1) pointwise convolution с 1x1-свёрткой с активацией ReLU6 ($\text{ReLU6}(x) = \min(\max(0, x), 6)$), который создает отображение входного тензора в пространстве большой размерности и расширяет каналы, 2) depthwise convolution (поканальная свёртка) с активацией ReLU6 значительно уменьшает количество параметров, 3) 1x1-свёртка с линейной функцией активации понижает число каналов. Bottleneck layers есть с шагом (stride) 1 и 2. Блок с шагом = 2 нужен для снижения пространственной размерности тензора и не имеет остаточных соединений.

Основная идея - нелинейность уничтожает информацию в канале. Получив ноль после применения ReLU ($\text{ReLU}(x) = \max(0, x)$), нельзя сказать, какое отрицательное число было. А используя линейные слои, модель сохраняет больше информации и улучшает свою способность захватывать мелкие детали.

⁵ <https://arxiv.org/abs/1801.04381>

Следующая архитектура — MobileNetv3-Small⁶, была выбрана для сравнения с результатами модели MobileNetv2, как изменения в архитектуре v3 могут повлиять на результат, насколько улучшат его или нет. MobileNetv3 также предобучена на данных ImageNet (1000 классов), вес модели 9.8M.

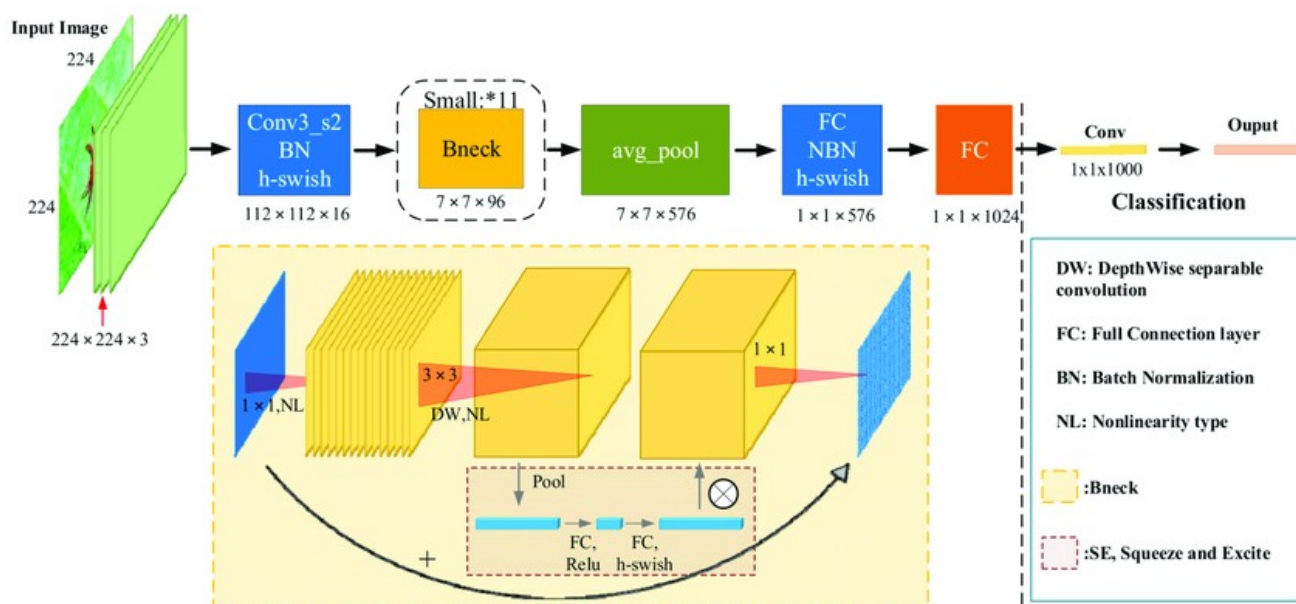


Рис 7. Архитектура MobileNetv3

MobileNetv3 во многом была создана, благодаря методам [MnasNet](https://arxiv.org/abs/1905.02244)⁷ (Platform-Aware Neural Architecture Search for Mobile, поиск грубой архитектуры, которая использует обучение с подкреплением для выбора оптимальной конфигурации из дискретного набора вариантов) и [NetAdapt](https://arxiv.org/pdf/1804.03230.pdf)⁸ (Platform-Aware Neural Network Adaptation for Mobile Applications, алгоритм, который автоматически адаптирует предварительно обученную глубокую нейросеть к мобильной платформе с учётом бюджета ресурсов, плюс эта технология небольшими порциями сокращает недостаточно используемые каналы активации). Оба метода учитывают время работы на мобильных устройствах для выбора оптимальных по соотношению качество/скорость вариантов.

В модель добавлены Squeeze-and-Excitation блоки⁹ (рис.7 и 8), которые заменяют классическую сигмовидную функцию кусочно-линейной аппроксимацией. Блок взвешивается «толстые» тензоры, на которых применяется нелинейность. В блоке: снижение количества фильтров с помощью hard-swish ($h\text{-swish}(x) = x * ((\text{ReLU6}(x+3)) / 6)$). С 32 фильтров с ReLU до 16 фильтров с h-swish, которая основана на дорогостоящей в вычислительном отношении для мобильных устройств функции нелинейности Swish (или SiLU).

Также в модели представили новый сокращённый последний этап в конце сети, рис. 9. Отбросили три дорогостоящих слоя без потери точности.

⁶ <https://arxiv.org/abs/1905.02244>

⁷ <https://research.google/pubs/mnasnet-platform-aware-neural-architecture-search-for-mobile/>

⁸ <https://arxiv.org/pdf/1804.03230.pdf>

⁹ <https://paperswithcode.com/method/squeeze-and-excitation-block>

Squeeze-And-Excite (SE) layer

Notes: Tensors' shapes are for illustration purposes. They vary according to the layer's params. The first dimension of tensors' shapes is always the batch size.

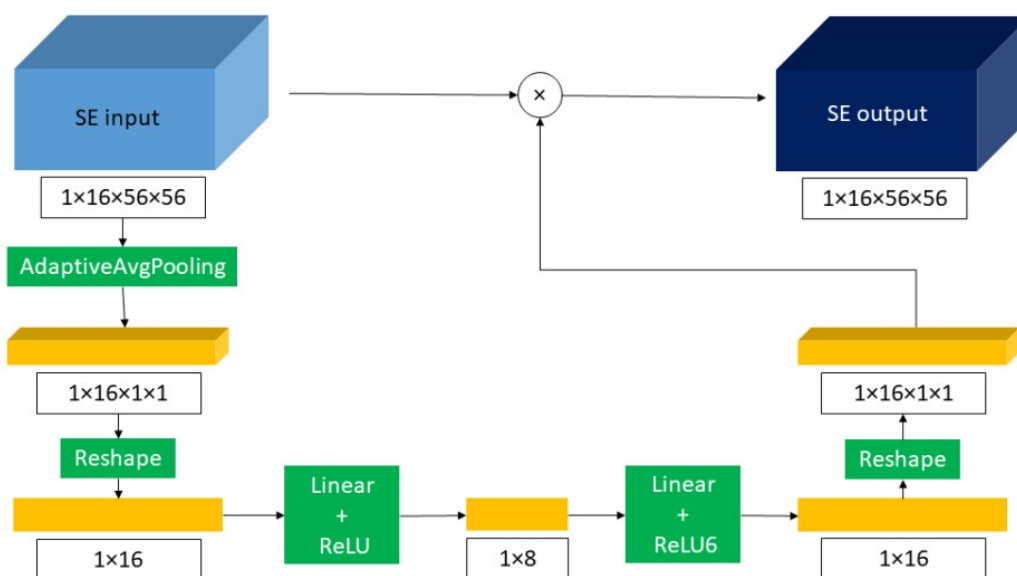


Рис. 8 Squeeze-and-Excitation блок

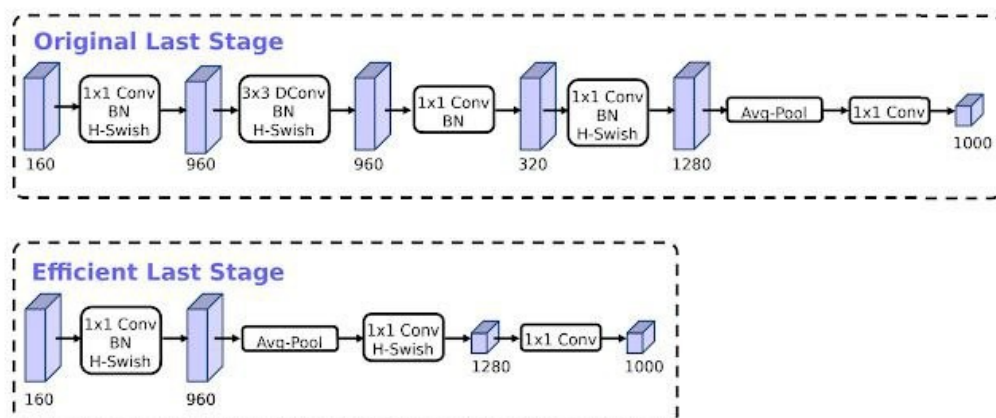


Рис. 9 Последний этап модели (Efficient last stage)

Основная идея Squeeze-and-Excitation (SE) — блоки SE явно моделируют взаимоотношения каналов и взаимозависимости каналов. Также повторно калибруют карты признаков с помощью операции «сжатия» (squeeze) - average pooling до 1 числа на каждый фильтр, за которой следует операция «активизации» (excitation), в которой используется серия линейных слоёв плюс нелинейности (ReLU, ReLU6), два полносвязанных слоя (первый уменьшает размерность, второй увеличивает её до размера SE input (рис.8)) и умножается результат с исходными входными данными (SE input). Блоки могут быть подключены к любой архитектуре свёрточных нейросетей и требуют минимальных вычислительных затрат. Блоки могут повысить производительность задач классификации и обнаружения объектов.

И последняя модель efficientnetv2-small¹⁰, которая была выбрана после прочтения одного исследования, которое было проведено М. Утробиным и опубликовано им на habr.com в мае

¹⁰ <https://arxiv.org/abs/2104.00298>

2022 года с названием «Большое сравнение 400 нейронных сетей для задачи классификации на более 8000 классов»¹¹. Целью его исследования было сравнение большого количества существующих архитектур предобученных сетей и их результатов для решения задач классификации изображений. Автор делает вывод, что по точности победила предобученная модель из группы EfficientNet (Топ-1).

Модель EfficientNetv2_small обучалась на данных ImageNet21k (21841 классов), вес модели 82.7М. Исследователи говорят, что модели EfficientNetV2 обучаются намного быстрее, чем современные модели, но при этом они в 6,8 раз меньше. Это новое семейство свёрточных сетей, для разработки которого используют комбинацию поиска и масштабирования нейронной архитектуры с учётом обучения для совместной оптимизации скорости обучения и эффективности параметров. Основная идея - комплексное масштабирование сети в нескольких измерениях (входное разрешение, глубина, ширина) с помощью составного коэффициента. Представлен новый модуль под названием Fused-MBConv, который применяется вместе с блоками MBConv блоками (блоки bottlenecks blocks вместе с блоками Squeeze-and-Excitation, как в модели MobileNetv3). Также используется прогрессивная техника обучения (Progressive Learning Technique)¹² для повышения скорости и точности обучения. Рис. 10

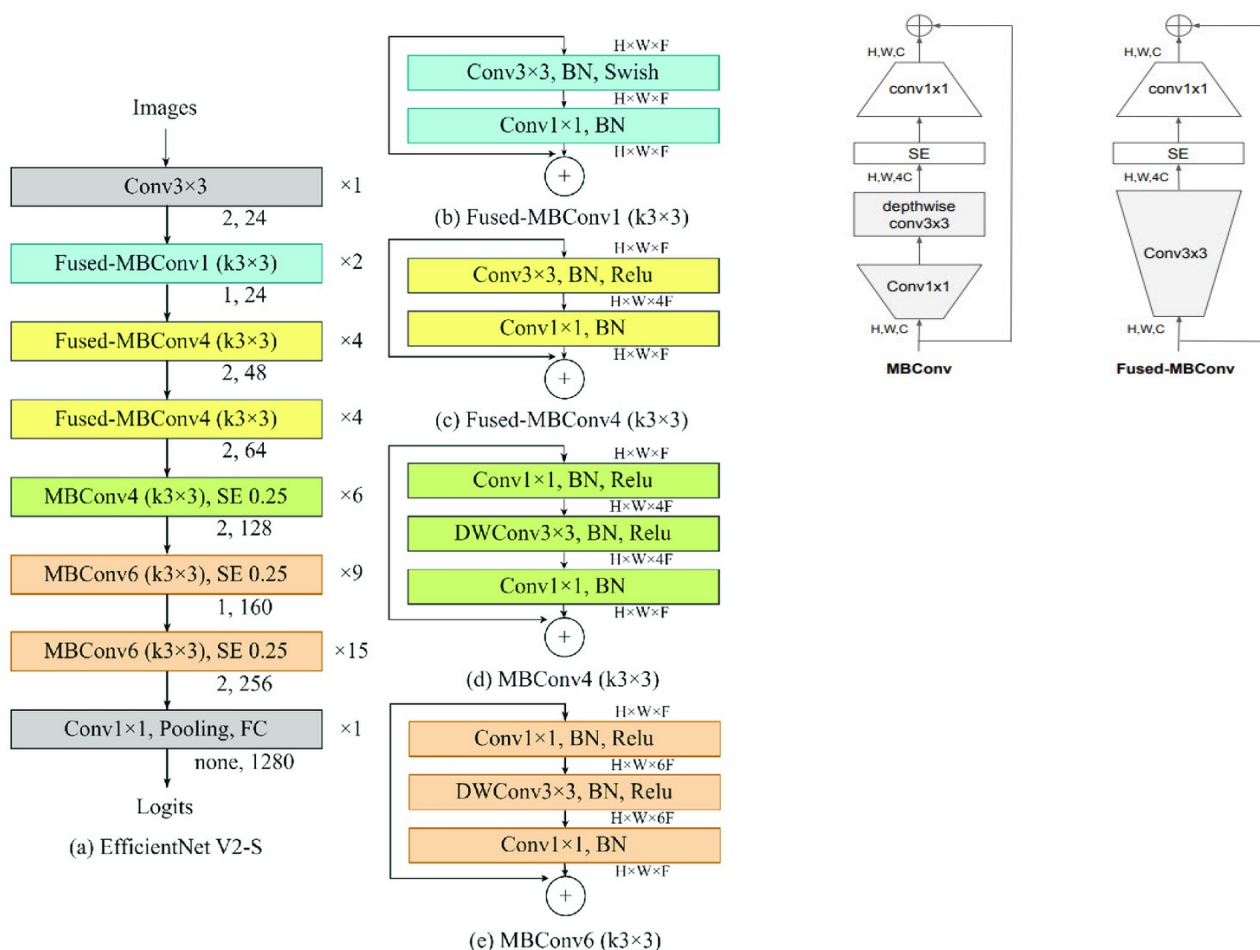


Рис. 10 Блоки и архитектура EfficientNetV2-S

Прогрессивное обучение — это постепенное увеличение размера изображения во время обучения, что приводит к падению точности и для компенсации этого, адаптивно настраивается регуляризация (например, исключение и увеличение данных) для достижения как быстрого обучения, так и хорошей точности.

¹¹ <https://habr.com/ru/articles/666314/>

¹² <https://arxiv.org/abs/1609.00085>

Основные отличия от моделей EfficientNet: широко используются как MBConv, так и Fused-MBConv; используется меньший коэффициент расширения для MBConv, так как они требуют меньших затрат по памяти; размеры ядра 3x3, но увеличение слоёв, чтобы компенсировать уменьшение рецептивного поля, возникающего из-за меньшего размера ядра; удалён последний этап stride-1.

МЕТОДИКА РЕАЛИЗАЦИИ

Проведённые эксперименты

Для решения поставленной задачи было проведено 7 экспериментов на трёх предобученных моделях. Использовала настройку fine-tuning с дополнительными слоями (функция активации ReLu, линейные слои), так как по результатам первого эксперимента с моделью mobilenetv2, с настройкой features extraction, на данных без аугментации модель не обучилась (Рис. 11). Результаты: потеря на train: 11.392 и на test: 11.602, точность на train: 0.12% и на test: **0.06%**.

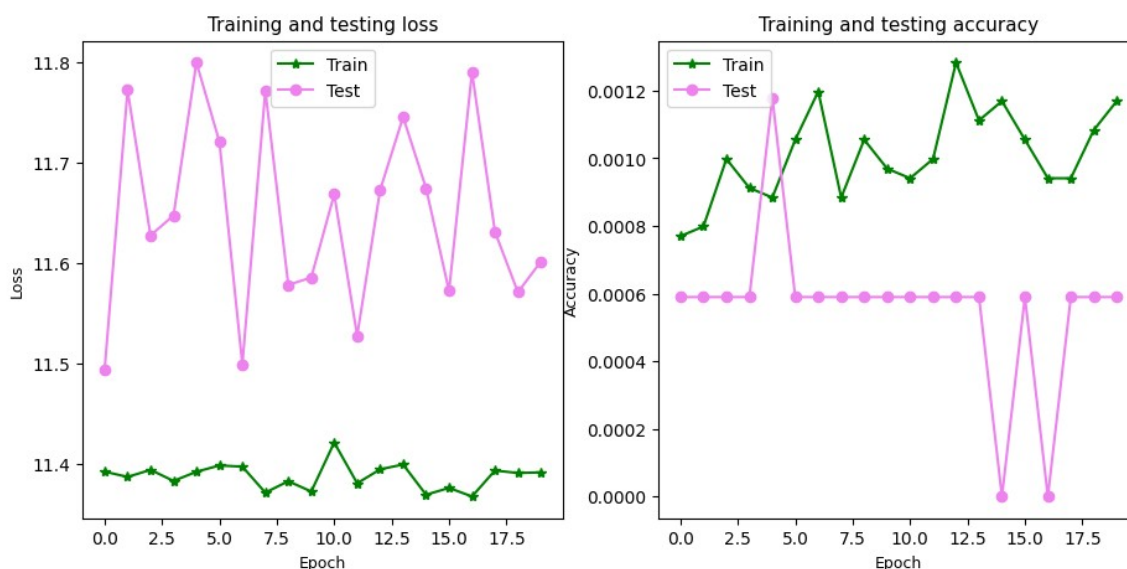


Рис. 11 Графики результатов потери и точности обучения модели mobilenetv2 (features extraction) без аугментации

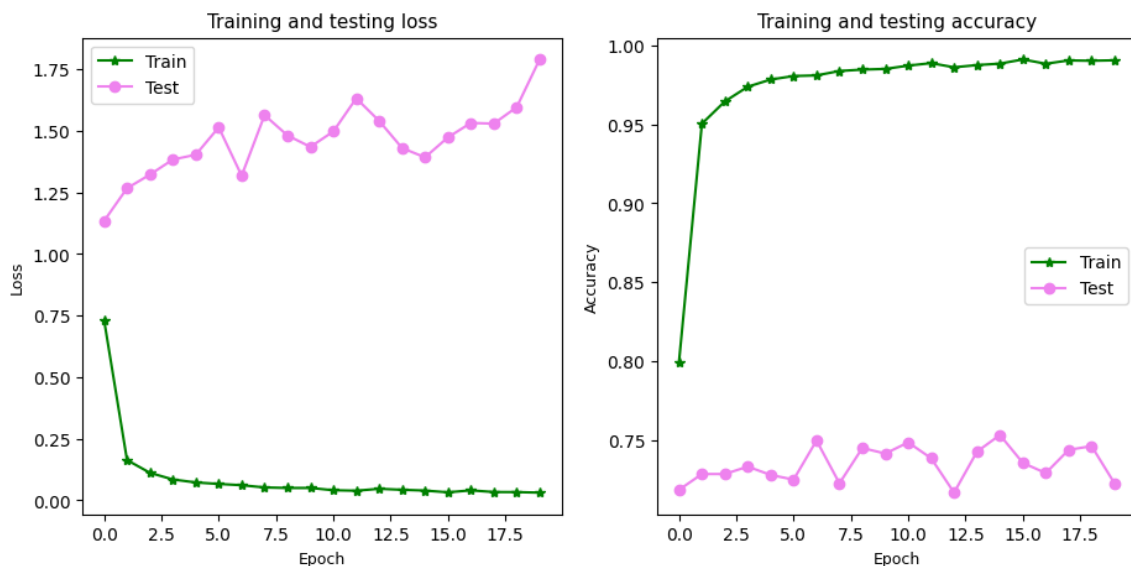


Рис. 12 Графики результатов потери и точности обучения модели mobilenetv2 без аугментации

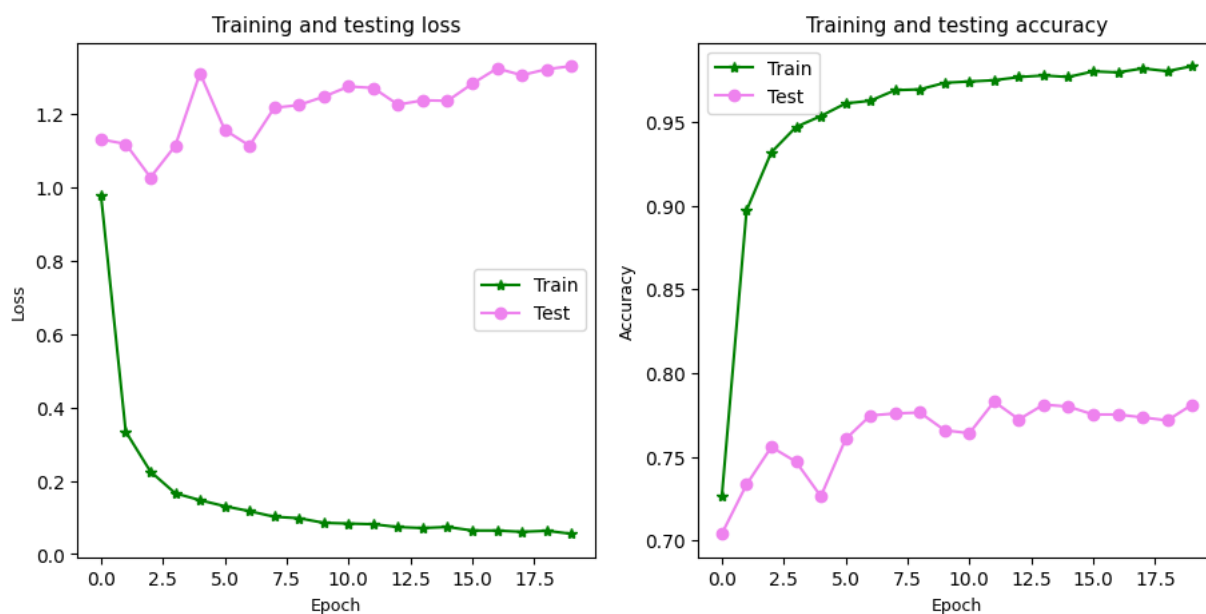


Рис. 13 Графики результатов потери и точности обучения модели mobilenetv2 с аугментацией

Обучение mobilenetv2 на данных без аугментации (Рис.12): потеря на train: 0.031 и на test: 1.790, точность на train: 99.04% и на test: **72.29%** и с применением аугментации (Рис.13): потеря на train: 0.055 и на test: 1.329, точность на train: 98.32% и на test: **78.12%**. Среднее время обучения 1 эпохи — около 8.5 минут. Как видно на графике на тестовых данных колебания, потеря растёт. На графике (рис.13) колебаний меньше. Точность на тренировочных данных упала на данных с аугментацией, а вот точность на тестовых данных, наоборот, выросла. Можно сделать вывод, что аугментация снизила немного переобучение и улучшила результат на тесте с 72.29% до 78.12%.

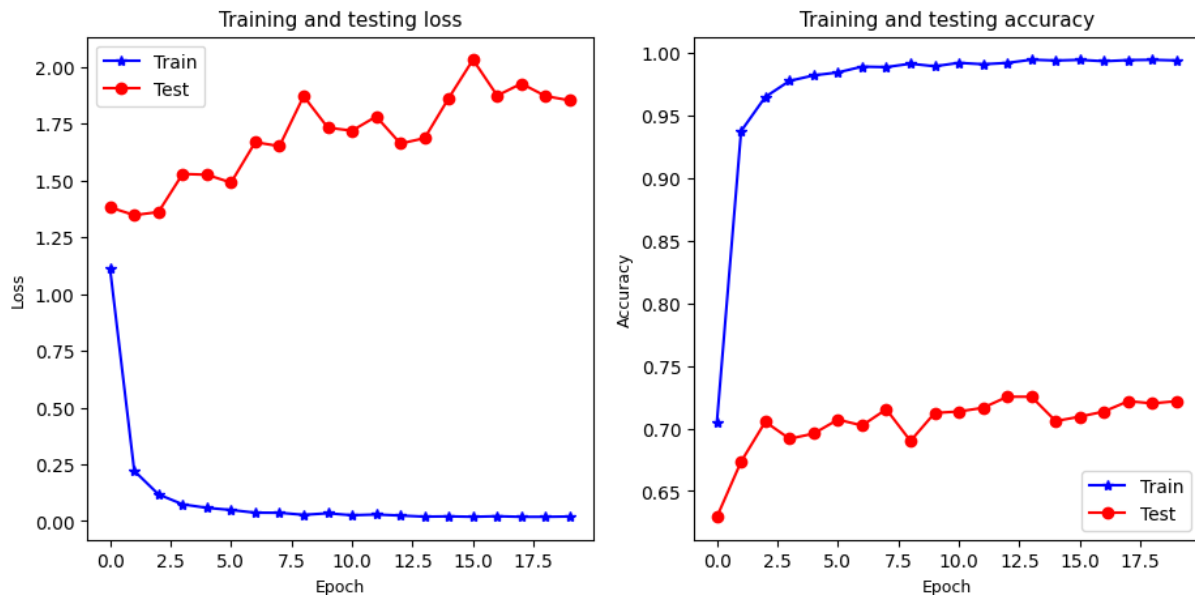


Рис. 14 Графики результатов потери и точности обучения модели mobilenetv3 без аугментации

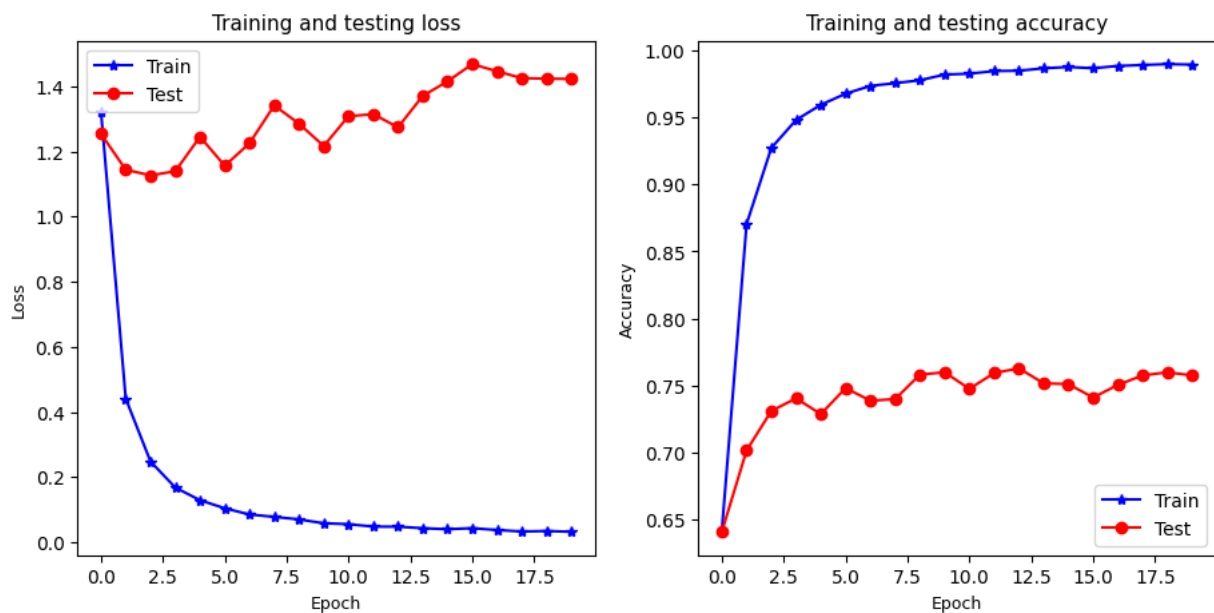


Рис.15 Графики результатов потери и точности обучения модели mobilenetv3 с аугментацией

Далее обучение mobilenetv3 на данных без аугментации (Рис.14): потеря на train: 0.020 и на test: 1.853, точность на train: 99.38% и на test: **72.18%** и с аугментацией (Рис.15): потеря на train: 0.032 и на test: 1.423, точность на train: 98.92% и на test: **75.76%**. Среднее время обучения 1 эпохи 9 минут. На графиках те же колебания, на графике (Рис.15) они немного сглаженные. На последних эпохах графики выравниваются. И здесь аугментация помогла снизить переобучение, пусть немного, и улучшить точность с 72.18% до 75.76%.

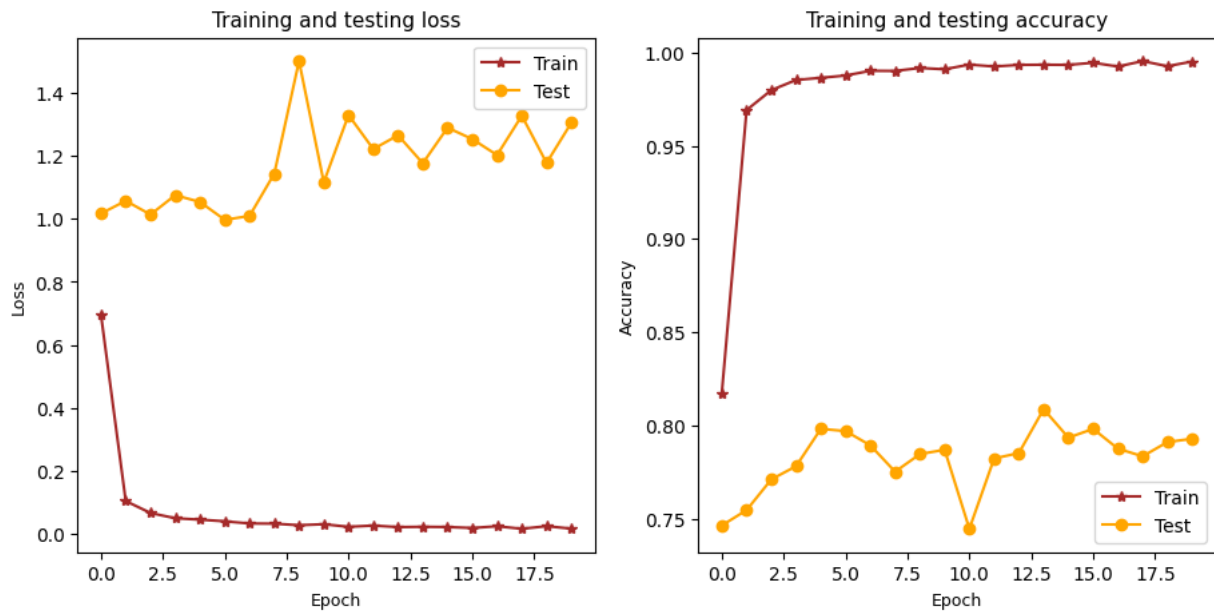


Рис. 16 Графики результатов потери и точности обучения модели efficientnetv2_s без аугментации

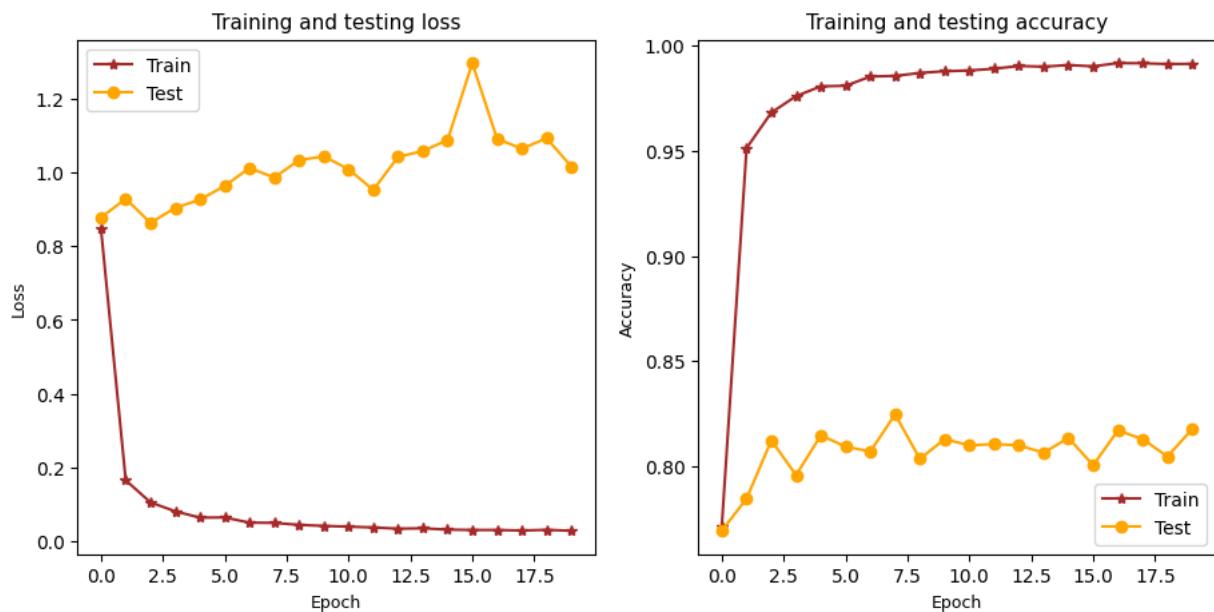


Рис. 17 Графики результатов потери и точности обучения модели efficientnetv2_s с аугментацией

Обучение efficientnetv2_s на данных без аугментации (Рис.16): потеря на train: 0.017 и на test: 1.305, точность на train: 99.51% и на test: **79.29%** и на данных с аугментацией (Рис.17): потеря на train: 0.017 и на test: 1.305, точность на train: 99.12% и на test: **81.76%**. Среднее время обучения 1 эпохи 9.5 минут. Графики похожи колебаниями. На графике (Рис.17) потеря на последних эпохах стала падать. Точность на данных с аугментацией выросла с 79.29% до 81.76%.

Полученные результаты

В таблице 1 представлены результаты экспериментов на тестовых данных. Лучший результат показала модель efficientnetv2_s, обученная на данных с применением аугментации.

Сравнивая результаты моего исследования и исследования, которое было проведено в 2021 г. на этом же наборе данных (информация в «Анализ данных», стр.4), на архитектуре RegNet, на 20 эпохах, применение 1 аугментации, Cross-entropy loss, Adam, размерность 224*224 с точностью 77.75%.

Модели	Потеря	Точность (%)
mobilenet_v2	1.790	72.29
mobilenet_v2_aug	1.329	78.12
mobilenet_v3_s	1.853	72.18
mobilenet_v3_s_aug	1.423	75.76
efficientnet_v2_s	1.305	79.29
efficientnet_v2_s_aug	1.016	81.76

Таблица 1

ВЫВОДЫ

В ходе проведённых экспериментов была обучена модель классификации изображений картин 50 известных художников. Данная модель efficientnetv2_small является предобученной моделью на данных ImageNet21k с настройкой fine-tuning. Достигнутая точность составила 81.76%, что является результатом превосходящим результат, заявленный в задаче к этой работе получить выше 80% на тестовой выборке.

Аугментация улучшила точность, помогла немного снизить переобучение, значит, применение аугментации оправданно и желательно.

Целью работы было научить модель предсказывать художника по картине, чтобы модель научилась «видеть» особенности техники каждого художника. Думаю, что цель достигнута именно в этой работе.

В качестве дальнейших экспериментов можно попробовать добавить dropout для уменьшения/устранения переобучения модели; применить к оптимизатору Adam какой-то из schedulers; или применить оптимизатор AdamW; для устранения дисбаланса классов применить другой подход; применить различные варианты аугментации, снизить или увеличить количество аугментаций; увеличить количество эпох (50, 100...), увеличить батч (128, 256...) и т. д.

СПИСОК ИСТОЧНИКОВ*

1. «Аналитический обзор методов оценки качества алгоритмов классификации в задачах машинного обучения». Авторы: А.А. Михайличенко, Институт математики, механики и компьютерных наук, Южный федеральный университет, Ростов-на-Дону, издание «Вестник АГУ» », № 4 (311) 2022.
2. Балансировка классов в torch с помощью WeightedRandomSampler
<https://traceback.ru/balance-classes-in-torch-with-weighted-random-sampler/>
3. Большое сравнение 400 нейронных сетей для задачи классификации на более 8000 классов
<https://habr.com/ru/articles/666314/>
4. Выбор слоя активации в нейронных сетях: как правильно выбрать для вашей задачи.
<https://habr.com/ru/articles/727506/>
5. Дисбаланс классов <https://alexanderdyakonov.wordpress.com/2021/05/27/imbalance/>
6. Искусственное искусство: как нейросети научились творить.
<https://www.forbes.ru/tehnologii/368515-iskusstvennoe-iskusstvo-kak-neyroseti-nauchilis-tvorit>
7. Искусство «больших данных»: как музеи и парки используют аналитику для устойчивого развития и привлечения аудитории.
<https://vc.ru/marketing/143420-iskusstvo-bolshih-dannyh-kak-muzei-i-parki-ispolzuyut-analitiku-dlya-ustoychivogo-razvitiya-i-privlecheniya-auditorii>
8. Использование свёрточных нейронных сетей для классификации изображений.
<https://cyberleninka.ru/article/n/ispolzovanie-svortchnyh-neyrosetey-dlya-klassifikatsii-izobrazheniy/viewer>
9. Как применяются нейросети в современном мире? <https://vc.ru/u/1560497-neyronochka/634194-kak-primenyayutsya-neyroseti-v-sovremennom-mire?ysclid=lrwbesd2tu342423967>
10. Обзор научной публикации: Классификация картин при помощи нейронных сетей
<https://datareview.info/article/obzor-nauchnoy-publikatsii-klassifikatsiya-kartin-pri-pomoshhi-neyronnyih-setey/>
11. Обзор свёрточных нейронных сетей для задачи классификации изображений.
<https://cyberleninka.ru/article/n/obzor-svyortchnyh-neyronnyh-setey-dlya-zadachi-klassifikatsii-izobrazheniy/viewer>
12. «Применение свёрточных нейронных сетей в литологическом описании керна». Авторы: И.Д. Муртазаев, Н.Ж. Конысов, Н.Б. Салиев, издание «Вестник нефтегазовой отрасли Казахстана», № 2 (3) 2020, стр. 20-27.
13. Программа научилась классифицировать живопись по стилям и авторам.
<https://nplus1.ru/news/2015/05/13/are-you-sure-this-is-monet>
14. «Распознавание художника по картине». Авторы: И.И. Евтушенко, А.В. Вечур, издание «Бионика интеллекта», № 1 (96) 2021, № 1 (96). стр. 96–102
15. Стили и техники в живописи. <https://art-kvartira.ru/stati/stili-i-tehniki-v-zhivopisi/>
16. A visual deep-dive into the building blocks of MobileNetV3 <https://francescopochetti.com/a-visual-deep-dive-into-the-building-blocks-of-mobilenetv3/>
17. Address class imbalance easily with Pytorch
<https://towardsdatascience.com/address-class-imbalance-easily-with-pytorch-bb540497d2a6>
<https://towardsdatascience.com/address-class-imbalance-easily-with-pytorch-e2d4fa208627>
18. Cross-Entropy, Negative Log-Likelihood, and All That Jazz. <https://towardsdatascience.com/cross-entropy-negative-log-likelihood-and-all-that-jazz-47a95bd2e81>
19. Demystifying PyTorch's WeightedRandomSampler by example [Demystifying PyTorch's WeightedRandomSampler by example | by Chris Hughes | Towards Data Science](https://towardsdatascience.com/demystifying-pytorchs-weightedrandomsampler-by-example-by-chris-hughes-towards-data-science)
20. EfficientNetV2: Faster, Smaller, and Higher Accuracy than Vision Transformers.
<https://towardsdatascience.com/efficientnetv2-faster-smaller-and-higher-accuracy-than-vision-transformers-98e23587bf04>

21. EfficientNetV2: Smaller Models and Faster Training. Review.
<https://mlpapersreview.wordpress.com/2021/04/03/review-efficientnetv2-smaller-models-and-faster-training/>
22. Image Classification on ImageNet
<https://paperswithcode.com/sota/image-classification-on-imagenet> и
<https://paperswithcode.com/task/image-classification>
23. Introducing the Next Generation of On-Device Vision Models: MobileNetV3 and MobileNetEdgeTPU <https://blog.research.google/2019/11/introducing-next-generation-on-device.html>
24. Inverted Residual Block <https://paperswithcode.com/method/inverted-residual-block>
25. Kaggle www.kaggle.com
26. Large-scale Classification of Fine-Art Paintings: Learning The Right Metric on The Right Feature,
<https://arxiv.org/pdf/1505.00855v1.pdf>
27. MobileNet: меньше, быстрее, точнее. <https://habr.com/ru/articles/352804/>
28. MobileNetV2: The Next Generation of On-Device Computer Vision Networks
<https://blog.research.google/2018/04/mobilenetv2-next-generation-of-on.html>
29. MobileNet v1-v3 <https://telegra.ph/MobileNet-v1-v3-11-24>
30. Squeeze-and-Excitation Networks <https://towardsdatascience.com/squeeze-and-excitation-networks-9ef5e71eacd7>
31. «Survey on deep learning with class imbalance.» Авторы: Justin M. Johnson, Taghi M. Khoshgoftaar, издание «Journal of Big Data», номер статьи 27 (2019).
<https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0192-5>

Код работы

<https://colab.research.google.com/drive/1Gx2gwGOSEb5OuCg13a3GBXkoIDnOWb3L#scrollTo=Injnr eYi9n9V>

* Для некоторых статей нужно подключение к сервису VPN.